arXiv:2410.10989v3 [cs.LG] 24 Jan 2025

Liger Kernel: Efficient Triton Kernels for LLM Training

Pin-Lun Hsu, Yun Dai, Vignesh Kothapalli, Qingquan Song, Shao Tang,
Siyu Zhu, Steven Shimizu, Shivam Sahni, Haowen Ning and Yanning Chen

LinkedIn Inc

Abstract

Training Large Language Models (LLMs) efficiently at scale presents a formidable challenge, driven by their ever-increasing computational demands and the need for enhanced performance. In this work, we introduce `Liger-Kernel`, an open-sourced set of Triton kernels developed specifically for LLM training. With kernel optimization techniques like kernel operation fusing and input chunking, our kernels achieve on average 20% increase in training throughput and a 60% reduction in GPU memory for popular LLMs compared with HuggingFace implementations. In addition, `Liger-Kernel` is designed with modularity, accessibility and adaptability in mind, catering to casual and expert users. Comprehensive benchmarks and integration tests are built-in to ensure compatibility, performance, correctness and convergence across diverse computing environments and model architectures. The source code is available under a permissive license https://github.com/linkedin/Liger-Kernel.

1 Introduction

Scaling Large Language Model (LLM) training (Vaswani, 2017; Wei et al., 2022; Brown et al., 2020; Team et al., 2023; Touvron et al., 2023; Dubey et al., 2024) relies heavily on the stability of compute infrastructure and is susceptible to efficiency bottlenecks. Host/device memory management and latency-bandwidth trade-offs for tensor operations are central to the efficiency issues. However, beyond algorithmic scaling strategies, the true potential for optimization lies in fusing operations at the GPU kernel level, which minimizes memory copying and maximizes parallel efficiency. These last-mile kernel-level optimizations are crucial because any gains at this level are amplified by the inherent parallelism of GPUs, making them indispensable for improving overall training performance. Despite recent advancements in hardware and software usability for distributed training, optimizing the training process remains a highly complex and specialized task - which requiring not only a deep understanding of both LLM algorithms and hardware architectures but also significant time and financial investments.

To address these challenges, we present `Liger-Kernel`, an open-source library of efficient Triton kernels (Tillet et al., 2019) for LLM training. `Liger-Kernel` enhances the efficiency and scalability of LLM training through a highly flexible and user-friendly interface. It streamlines complex tensor operations, minimizes computational overheads with kernel fusions (Dao et al., 2022) and seamlessly integrates with diverse computing environments. Novice users can improve LLM training efficiency with a few lines of code, while advanced users can customize their model with modular components and adaptive layer configurations to suit their needs. `Liger-Kernel` requires minimal dependencies, i.e., PyTorch (Zhao et al., 2023) and Triton. `Liger-Kernel` supports multiple distributed frameworks such as PyTorch FSDP, DeepSpeed ZeRO (Rasley et al., 2020) and ZeRO++(Wang et al., 2023; Dai et al., 2024), ensuring broad compatibility and performance optimization across various hardware platforms.

2 Preliminaries

Eager mode execution in PyTorch (Paszke et al., 2019) provides a smooth development and debugging experience when authoring model code. However, step-by-step execution of PyTorch operations entails extra computational overheads, including function call stack, dispatching, and CUDA kernel launch latencies.

In addition, materializing every intermediate activation for backward pass also introduces significant GPU memory usage. The majority of the efforts for addressing this issue have focused on model compilation and algorithmic operation fusion. Recently, more practitioners are implementing custom operation fusion in the Triton language (Tillet et al., 2019) to replace native PyTorch execution of model code.

2.1 Model Compiler

Model compilers transform high-level model descriptions (for example, `torch.nn.Module`) into optimized, low-level code that can be executed more efficiently, particularly on specialized hardware such as GPUs. Examples of such compilers include `torch.compile` (Ansel et al., 2024), TVM (Chen et al., 2018), XLA (Sabne, 2020), and nvFuser. `torch.compile` is the latest PyTorch-native model compilation feature introduced in PyTorch 2.0. Its frontend just-in-time (JIT) captures the computational graph and converts python-level operations into an intermediate representation (IR). Its backend performs low-level optimizations on the IR and translates into high-performance code in Triton for GPUs and C++ with OpenMP for CPUs. Apache TVM provides a unified intermediate representation for various hardware platforms, aiming to bridge the gap between high-level deep learning frameworks and diverse deployment targets. XLA, developed by Google, is designed to optimize TensorFlow (Abadi et al., 2016) and JAX (Frostig et al., 2018) based training workflows. It performs operation fusion, layout optimization, and kernel generation tailored to the target hardware. nvFuser is a PyTorch-specific JIT compiler developed by NVIDIA. It is especially capable of generating optimized CUDA code tailored to the specific GPU, taking advantage of the GPU architecture's capabilities, such as memory hierarchy, parallelism, and instruction-level optimizations.

2.2 An Algorithmic Perspective of Operation Fusion

The cornerstone of `Liger-Kernel`'s design is operation fusion. The main goal of the custom operation fusion is to mitigate the bottleneck arises between the high-bandwidth memory (HBM) and the shared memory (SRAM) for frequent memory copy. Each streaming multiprocessor (SM) needs fast access to data to execute multiple threads in parallel, but HBM, while large, is significantly slower than SRAM. This mismatch can lead to delays, where the processing cores sit idle, waiting for data to transfer from HBM to the faster, more limited SRAM. This becomes more severe in the context of deep learning models, especially those with large matrices (like in transformers) and numerous operations[1]. Operation fusion combines several standalone GPU operations into a single one to avoid the per-op time and memory overhead in step-by-step execution mentioned at the beginning of Section 2. From an algorithmic perspective, operation fusion techniques like FlashAttention (Dao et al., 2022; Dao, 2023) offer the advantage of optimizing specific computational patterns inherent to the algorithm itself, enabling more precise and tailored performance improvements compared to the broader, more generalized optimizations performed by model compilers. FlashAttention, for instance, optimizes the attention computation in transformer models by leveraging GPU memory hierarchies, reducing memory complexity from quadratic to linear. It splits the attention computation into smaller blocks that fit into the GPU on-chip SRAM, avoiding the need to materialize the full attention matrix and redundant memory accesses to the slower GPU high-bandwidth memory (HBM). FlashAttention-2 further improves this approach by reducing register spilling and enhancing parallelism across attention heads. These innovations collectively result in significant speedups and memory savings for attention computations, particularly for long sequence lengths.

2.3 Custom Operation Fusion with Triton

OpenAI's Triton is a programming language and compiler for high-performance GPU kernels with Python-like syntax (simpler than CUDA), making it easier to optimize deep learning operations without the complexity of low-level GPU programming. The JIT-compile nature of it also allows libraries and tools that use it to be more lightweight and portable. These features have increased the popularity of Triton for writing high-performance kernels for PyTorch on GPUs. `xFormers` (Lefaudeux et al., 2022) from Meta hosts interoperable and optimized Transformer building blocks implemented in Triton and CUDA and supports various

[1] Wen-Mei et al. (2022) provides more detailed strategies to alleviate this bottleneck and optimize GPU performance.

attention mechanisms. he FlashAttention repository[2], in addition to hosting the CUDA implementation of FlashAttention algorithms, also includes other Transformer building block implementations (such as layer norm, a fused implementation of linear layer and squared ReLU activation etc) in Triton and `torch.script`. Unsloth[3] from Unsloth AI re-implements popular LLMs (Touvron et al., 2023; Jiang et al., 2023; Abdin et al., 2024) and LoRA (Hu et al., 2021) adapter layer in Triton to support efficient LLM fine-tuning and fast inference. Similar to the tiling design in FlashAttention, EfficientCrossEntropy[4] fuses linear projection with CrossEntropy loss, and computes the loss in a block-wise manner to avoid materializing the entire logits tensor. `Liger-Kernel` draws inspiration and leverages code from some of the aforementioned projects as references. The details are presented in Section 3.2.

3 Liger Kernel

3.1 API Design

Ease of use is crucial for community adoption, and Liger kernels are designed to be accessible and straightforward. The guiding principle behind Liger's API design is to be the least disruptive to users' existing codebases while providing the flexibility needed for various levels of customization. Depending on the level of customization required, there are several ways to apply Liger kernels:

1. **Using AutoLigerKernelForCausalLM:** The simplest way to leverage Liger kernels is through the `AutoLigerKernelForCausalLM` class. This approach requires no model-specific patching API imports. If the model type is supported, the modeling code will be automatically patched by Liger.

```
from liger_kernel.transformers import AutoLigerKernelForCausalLM

model = AutoLigerKernelForCausalLM.from_pretrained("path/to/some/model")
```

2. **Applying Model-Specific Patching APIs:** For fine-grained control over the model code, users can leverage `Liger-Kernel`'s model-specific patching APIs. These APIs are versatile and can be used with various model architectures beyond causal language models, such as sequence classification.

```
from liger_kernel.transformers import apply_liger_kernel_to_llama

apply_liger_kernel_to_llama()
model = AutoModelForSequenceClassification.from_pretrained("/path/to/some/model")
```

3. **Composing Custom Models:** Advanced users can leverage individual Liger kernels (as required) to create their own custom models. For instance, the torch-like code below illustrates the creation of a `LigerTransformer` module, which leverages `LigerLayerNorm` to implement the layer normalization functionality and `LigerCrossEntropyLoss` to create the loss function.

```
import torch
from liger_kernel.transformers import LigerLayerNorm, LigerCrossEntropyLoss

class LigerTransformer(torch.nn.Module):
    def __init__(self, hidden_dim, *args, **kwargs):
        super().__init__()
        # create attn, mlp blocks or any custom operation
        ...
        # use Triton-optimized LigerLayerNorm
        self.layer_norm = LigerLayerNorm(hidden_dim)

    def forward(self, x):
        # forward pass of the model
        ...
```

[2] github.com/dao-ailab/flash-attention
[3] https://github.com/unslothai/unsloth
[4] https://github.com/mgmalek/efficient_cross_entropy

```
# use the Triton-optimized LigerCrossEntropyLoss
loss_fn = LigerCrossEntropyLoss()
```

These flexible options ensure that Liger kernels can be easily integrated into various workflows, promoting efficient training and deployment of LLMs.

3.2 Kernels

Throughout the discussion, vectors[5] and matrices are represented by bolded lowercase and uppercase letters, e.g., $\boldsymbol{x} \in \mathbb{R}^n$ and $\boldsymbol{W} \in \mathbb{R}^{m \times n}$. The all-ones vector is denoted as $\mathbf{1}_n \in \mathbb{R}^n$. Functions are applied to the variable element-wise, i.e., $f(\boldsymbol{x})_i = f(x_i)$. We use $\odot$ to denote the element-wise product between tensors, and $^\top$ to denote the matrix transpose.

In our kernel implementations, both input and output tensors are reshaped into two-dimensional matrices with the shape $(B \times T, H)$, where B is the batch size, T is the sequence length and H is the hidden dimension.

In each kernel, Triton parallelizes operations on each row of input[6]. Therefore, we focus on the mathematical operations given a row of input denoted as $\boldsymbol{x}$ and the corresponding output denoted as $\boldsymbol{y}$. In the backward pass, given a loss function $\mathcal{L}$, we use $\nabla_{\boldsymbol{y}}\mathcal{L}$ to denote the gradient back-propagated from $\mathcal{L}$ to $\boldsymbol{y}$.

RMSNorm. We fuse the normalization and scaling steps of the RMSNorm computation into a single Triton kernel[7]. Specifically, given the input $\boldsymbol{x} \in \mathbb{R}^n$ and the learnable parameters $\boldsymbol{\gamma} \in \mathbb{R}^n$, the output $\boldsymbol{y} \in \mathbb{R}^n$ is defined as (Zhang and Sennrich, 2019):

$$\boldsymbol{y} = \hat{\boldsymbol{x}} \odot \boldsymbol{\gamma}, \qquad \hat{\boldsymbol{x}} = \frac{\boldsymbol{x}}{\mathrm{RMS}(\boldsymbol{x})}, \tag{1}$$

where $\hat{\boldsymbol{x}} \in \mathbb{R}^n$ is the normalized input, $\mathrm{RMS}(\boldsymbol{x}) = \sqrt{\sum_i x_i^2/n + \epsilon}$ and ϵ is a small constant for numerical stability. In the backward pass, we have the gradient back-propagated to $\boldsymbol{x}$ and $\boldsymbol{\gamma}$ as

$$\begin{aligned} \nabla_{\boldsymbol{x}}\mathcal{L} &= \frac{1}{\mathrm{RMS}(\boldsymbol{x})}\left(\nabla_{\boldsymbol{y}}\mathcal{L} \odot \boldsymbol{\gamma} - \underbrace{\left[\hat{\boldsymbol{x}}^\top(\nabla_{\boldsymbol{y}}\mathcal{L} \odot \boldsymbol{\gamma})/n\right]}_{\text{a numerical value}} \hat{\boldsymbol{x}}\right), \\ \nabla_{\boldsymbol{\gamma}}\mathcal{L} &= \nabla_{\boldsymbol{y}}\mathcal{L} \odot \hat{\boldsymbol{x}}. \end{aligned} \tag{2}$$

Since the same $\boldsymbol{\gamma}$ is applied to all input vectors $\boldsymbol{x}$ in the same batch, the gradients need to be summed up.

LayerNorm. Similar to the RMSNorm, given the input $\boldsymbol{x} \in \mathbb{R}^n$, the learnable parameters $\boldsymbol{\gamma} \in \mathbb{R}^n$ and $\boldsymbol{\beta} \in \mathbb{R}^n$, the output $\boldsymbol{y} \in \mathbb{R}^n$ is defined as (Ba et al., 2016):

$$\boldsymbol{y} = \tilde{\boldsymbol{x}} \odot \boldsymbol{\gamma} + \boldsymbol{\beta}, \qquad \tilde{\boldsymbol{x}} = \frac{\boldsymbol{x} - \bar{\boldsymbol{x}}}{\mathrm{RMS}(\boldsymbol{x} - \bar{\boldsymbol{x}})}, \tag{3}$$

where $\tilde{\boldsymbol{x}} \in \mathbb{R}^n$ is the centered and normalized input, with $\bar{\boldsymbol{x}} = (\sum_i x_i/n)\,\mathbf{1}_n$. In the backward pass, we have the gradient back-propagated to $\boldsymbol{x}$, $\boldsymbol{\gamma}$ and $\boldsymbol{\beta}$ as

$$\begin{aligned} \nabla_{\boldsymbol{x}}\mathcal{L} &= \frac{1}{\mathrm{RMS}(\boldsymbol{x} - \bar{\boldsymbol{x}})}\left(\nabla_{\boldsymbol{y}}\mathcal{L} \odot \boldsymbol{\gamma} - \underbrace{\left[\tilde{\boldsymbol{x}}^\top(\nabla_{\boldsymbol{y}}\mathcal{L} \odot \boldsymbol{\gamma})/n\right]}_{\text{a numerical value}} \tilde{\boldsymbol{x}} - \frac{1}{n}\left[(\nabla_{\boldsymbol{y}}\mathcal{L})^\top\boldsymbol{\gamma}\right]\mathbf{1}\right), \\ \nabla_{\boldsymbol{\gamma}}\mathcal{L} &= \nabla_{\boldsymbol{y}}\mathcal{L} \odot \tilde{\boldsymbol{x}} \\ \nabla_{\boldsymbol{\beta}}\mathcal{L} &= \nabla_{\boldsymbol{y}}\mathcal{L}. \end{aligned} \tag{4}$$

[5] Vectors are assumed to be column vectors unless otherwise specified.

[6] We compute the number of warps based on the block size, which is dependent upon the size of each row. We reuse the `calculate_settings` function from https://github.com/unslothai/unsloth/blob/main/unsloth/kernels/utils.py.

[7] The implementation is referenced the code from https://github.com/unslothai/unsloth/blob/main/unsloth/kernels/rms_layernorm.py and https://triton-lang.org/main/getting-started/tutorials/05-layer-norm.html.

Since the same $\boldsymbol{\gamma}$ and $\boldsymbol{\beta}$ are applied to all input vectors $\boldsymbol{x}$ in a batch, the gradients need to be summed up[8].

RoPE. We fuse the query and key rotation embedding computation into a single kernel to reduce overheads. For each rotary position embedding computation, given the input $\boldsymbol{x} \in \mathbb{R}^d$, the token position m and the rotation matrix $\boldsymbol{R}^d_{\Theta,m} \in \mathbb{R}^{d\times d}$, the output $\boldsymbol{y} \in \mathbb{R}^d$ is

$$\boldsymbol{y} = \boldsymbol{R}^d_{\Theta,m}\boldsymbol{x}. \tag{5}$$

Our implementation of RoPE assumes a rotation matrix in the form of HuggingFace model[9] instead of the rotation matrix described in Su et al. (2023). Namely,

$$\boldsymbol{R}^d_{\Theta,m} = \begin{pmatrix} \cos m\theta_1 & 0 & \dots & 0 & -\sin m\theta_1 & 0 & \dots & 0 \\ 0 & \cos m\theta_2 & \dots & 0 & 0 & -\sin m\theta_2 & \dots & 0 \\ 0 & 0 & \dots & 0 & 0 & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \cos m\theta_{d/2} & 0 & 0 & \dots & -\sin m\theta_{d/2} \\ \sin m\theta_1 & 0 & \dots & 0 & \cos m\theta_1 & 0 & \dots & 0 \\ 0 & \sin m\theta_2 & \dots & 0 & 0 & \cos m\theta_2 & \dots & 0 \\ 0 & 0 & \dots & 0 & 0 & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \sin m\theta_{d/2} & 0 & 0 & \dots & \cos m\theta_{d/2} \end{pmatrix}$$

where the parameters Θ is model specific.

In the backward pass, we have

$$\nabla_{\boldsymbol{x}}\mathcal{L} = (\boldsymbol{R}^d_{\Theta,m})^\top \nabla_{\boldsymbol{y}}\mathcal{L}. \tag{6}$$

In the implementation, due to the sparsity of $\boldsymbol{R}^d_{\Theta,m}$, we adopt the efficient computation in Su et al. (2023).

SwiGLU. We fuse the element-wise operations in the SwiGLU computation into a single kernel. Given the input $\boldsymbol{x} \in \mathbb{R}^n$ and learnable parameters $\boldsymbol{W} \in \mathbb{R}^{m\times n}, \boldsymbol{V} \in \mathbb{R}^{m\times n}, \boldsymbol{b} \in \mathbb{R}^m$ and $\boldsymbol{c} \in \mathbb{R}^m$, the output $\boldsymbol{y} \in \mathbb{R}^m$ is defined as (Shazeer, 2020):

$$\begin{aligned} \boldsymbol{y} &= \mathrm{Swish}_{\beta=1}(\boldsymbol{W}\boldsymbol{x}+\boldsymbol{b}) \odot (\boldsymbol{V}\boldsymbol{x}+\boldsymbol{c}) \\ &= \mathrm{SiLU}(\boldsymbol{W}\boldsymbol{x}+\boldsymbol{b}) \odot (\boldsymbol{V}\boldsymbol{x}+\boldsymbol{c}), \end{aligned} \tag{7}$$

where $\mathrm{SiLU}(z) = z\sigma(z)$ and $\sigma(z) = (1+\exp(-z))^{-1}$ is the sigmoid function. We only consider the $\beta = 1$ case here where Swish degenerates to SiLU, which aligns with the implementation of existing supported HuggingFace LLMs. Denote the values $\boldsymbol{x_1} = \boldsymbol{W}\boldsymbol{x}+\boldsymbol{b} \in \mathbb{R}^m$ and $\boldsymbol{x_2} = \boldsymbol{V}\boldsymbol{x}+\boldsymbol{c} \in \mathbb{R}^m$, we implement the kernel to compute the forward pass as

$$\boldsymbol{y}(\boldsymbol{x_1}, \boldsymbol{x_2}) = \mathrm{SiLU}(\boldsymbol{x_1}) \odot \boldsymbol{x_2}. \tag{8}$$

Recall $\nabla_{\boldsymbol{y}}\mathcal{L}$ as the gradient back-propagated from $\mathcal{L}$ to $\boldsymbol{y}$. In the backward pass, we have

$$\begin{aligned} \nabla_{\boldsymbol{x_1}}\mathcal{L} &= \nabla_{\boldsymbol{y}}\mathcal{L} \odot [\sigma(\boldsymbol{x_1}) + \mathrm{SiLU}(\boldsymbol{x_1}) \odot (1-\sigma(\boldsymbol{x_1}))] \odot \boldsymbol{x_2}, \\ \nabla_{\boldsymbol{x_2}}\mathcal{L} &= \nabla_{\boldsymbol{y}}\mathcal{L} \odot \mathrm{SiLU}(\boldsymbol{x_1}). \end{aligned} \tag{9}$$

[8]The efficient aggregation is non-trivial and three variants are benchmarked: plain aggregation in pytorch, two-stage aggregation from https://github.com/Dao-AILab/flash-attention/blob/main/flash_attn/ops/triton/layer_norm.py and atomic based aggregation in https://triton-lang.org/main/getting-started/tutorials/05-layer-norm.html. The latter two approaches perform much better than the vanilla aggregation and the second approach is currently adopted.

[9]https://github.com/huggingface/transformers/blob/v4.44.2/src/transformers/models/llama/modeling_llama.py#L253

GeGLU. Similar to SwiGLU, we fuse the element-wise operations. Given the input $\boldsymbol{x} \in \mathbb{R}^n$ and learnable parameters $\boldsymbol{W} \in \mathbb{R}^{m \times n}, \boldsymbol{V} \in \mathbb{R}^{m \times n}, \boldsymbol{b} \in \mathbb{R}^m$ and $\boldsymbol{c} \in \mathbb{R}^m$, the output $\boldsymbol{y} \in \mathbb{R}^m$ is defined as (Shazeer, 2020):

$$\boldsymbol{y} = \text{GELU}(\boldsymbol{W}x + \boldsymbol{b}) \odot (\boldsymbol{V}x + \boldsymbol{c}), \tag{10}$$

where we use the `tanh` approximation of GELU (Hendrycks and Gimpel, 2016). Formally,

$$\text{GELU}(z) \approx 0.5z \left(1 + \tanh\left[\sqrt{2/\pi}\left(z + 0.044715z^3\right)\right]\right). \tag{11}$$

Similar to SwiGLU, denote the values $\boldsymbol{x_1} = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b} \in \mathbb{R}^m$ and $\boldsymbol{x_2} = \boldsymbol{V}\boldsymbol{x} + \boldsymbol{c} \in \mathbb{R}^m$. The forward pass can be computed as:

$$\boldsymbol{y}(\boldsymbol{x_1}, \boldsymbol{x_2}) = \text{GELU}(\boldsymbol{x_1}) \odot \boldsymbol{x_2}. \tag{12}$$

In the backward pass, we have:

$$\begin{aligned} \nabla_{\boldsymbol{x_1}} \mathcal{L} &= \nabla_{\boldsymbol{y}} \mathcal{L} \odot \nabla_{\boldsymbol{x_1}} \text{GELU}(\boldsymbol{x_1}) \odot \boldsymbol{x_2}, \\ \nabla_{\boldsymbol{x_2}} \mathcal{L} &= \nabla_{\boldsymbol{y}} \mathcal{L} \odot \text{GELU}(\boldsymbol{x_1}), \end{aligned} \tag{13}$$

where

$$\begin{aligned} \nabla_{\boldsymbol{x_1}} \text{GELU}(\boldsymbol{x_1}) \approx & 0.5 \odot \left(1 + \tanh\left[\sqrt{2/\pi}\left(\boldsymbol{x_1} + 0.044715\boldsymbol{x_1}^3\right)\right]\right) \\ & + \sqrt{1/(2\pi)}\boldsymbol{x_1} \odot \left(1 - \tanh^2\left[\sqrt{2/\pi}\left(\boldsymbol{x_1} + 0.044715\boldsymbol{x_1}^3\right)\right]\right) \odot \left(1 + 0.134145\boldsymbol{x_1}^2\right). \end{aligned} \tag{14}$$

CrossEntropy (CE). We move the gradient computation to the forward function along with an inplace replacement of the logit tensor to avoid them being materialized simultaneously. We also adopt online softmax computation to compute the gradient on the fly. Given the input logits $\boldsymbol{x} \in \mathbb{R}^V$, where V is the vocabulary size, and target one-hot encoded label $\boldsymbol{t}$, the output probabilities are given as:

$$\boldsymbol{y} = \text{softmax}(\boldsymbol{x}), \tag{15}$$

and the cross-entopy loss is defined as $\mathcal{L} = -\sum_i t_i \log(y_i)$. The gradient back-propagated to $\boldsymbol{x}$ is given by:

$$\nabla_{\boldsymbol{x}} \mathcal{L} = \boldsymbol{y} - \boldsymbol{t}. \tag{16}$$

Additionally, we also employ the safe log operation to avoid numerical instabilities.

FusedLinearCrossEntropy (FLCE). The rapid expansion of vocabulary in recent LLMs aims to enhance token granularity and achieve more compact prompt representations. However, this progress has revealed a significant challenge: the materialization of logit tensors during CE loss computation consumes excessive memory. This issue has become a major bottleneck in LLM training, limiting our ability to increase batch sizes and extend prompt contexts. Take the Gemma model as an example, single GPU training with a batch size of 8 and sequence length of 4096, the 256k vocabulary size will result in a 16.8 GB logit tensor of precision bfloat16, causing a huge spike in the peak memory usage[10]. Although the CE loss kernel considers an in-place replacement of gradient and logits, preventing the double materialization of two large tensors, single logit tensor size is still prohibitive in many cases which motivates us to explore the chunked logit and gradient computation to amortize the memory consumption[11]. The main idea of FLCE is shown in Figure 1. The 3D hidden states (shifted already to align with their next ground truth tokens) are flattened into a 2D matrix by collapsing the batch size and sequence length dimensions into a single dimension. The linear projection head is applied sequentially on the chunked hidden states. The generated output logits are

[10] The memory usually peaks at the end of each forward pass right before the release of the activations in the backward pass.

[11] This is inspired from the GitHub discussions https://github.com/pytorch/pytorch/issues/124480 and the solution from https://github.com/mgmalek/efficient_cross_entropy

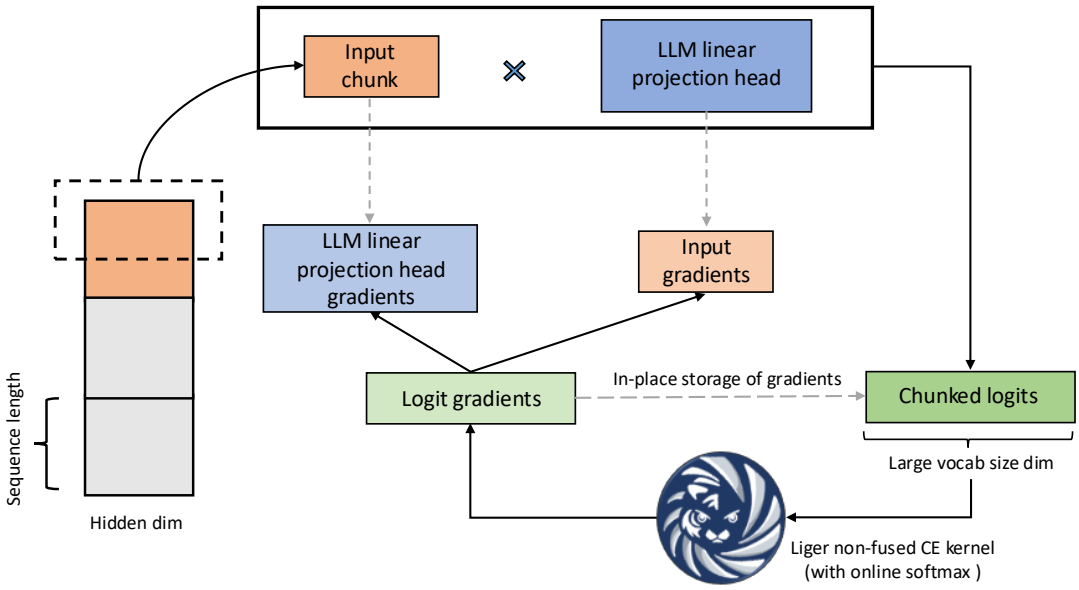


Figure 1: Fused Linear Cross Entropy.

passed to the non-fused Liger CE kernel to compute the partial loss and return the chunked logits gradient for deriving the chunked hidden states gradients and the accumulated projection head gradients.

$$\begin{aligned} \boldsymbol{x} &= \boldsymbol{W}^\top \boldsymbol{h}, \\ \nabla_{\boldsymbol{h}} \mathcal{L} &= \boldsymbol{W} \nabla_{\boldsymbol{x}} \mathcal{L}, \\ \nabla_{\boldsymbol{W}} \mathcal{L} &= \boldsymbol{h} (\nabla_{\boldsymbol{x}} \mathcal{L})^\top, \end{aligned} \tag{17}$$

where $\boldsymbol{W} \in \mathbb{R}^{H \times V}$ denotes the linear projection head weight given vocabulary size V. $\boldsymbol{h} \in \mathbb{R}^H$ indicates a single row of the flattened hidden state matrix $\boldsymbol{H} \in \mathbb{R}^{BT \times H}$. A single row can be viewed as the special case with a chunk size equal to 1. $\boldsymbol{x}$ represents the logits projected from $\boldsymbol{h}$, for which, we have derived its gradient based on (16). Since the same weight $\boldsymbol{W}$ is used for projecting all chunks, its final gradient needs to be summed up as $\nabla_{\boldsymbol{W}} \mathcal{L} = \sum_{\boldsymbol{h}} \boldsymbol{h} (\nabla_{\boldsymbol{x}} \mathcal{L})^\top$. Oftentimes, we can benefit from the compute-intensive behavior of the last layer projection, the overhead of block-wise matrix multiplications can be effectively compressed with delicate chunking on the tensor size to keep high GPU utilization with saturated operation time. In practice, we set the chunk size to be $2^{\lceil \log_2 \lceil \frac{BT}{\lceil V/H \rceil} \rceil \rceil}$ with an intuition on picking the chunk size to be closer to the hidden dimension size to balance the trade-off between memory allocation and processing speed.

Remark. We additionally scale the gradients of the chunked inputs and the projection layer weights with the ratio of $\frac{\text{chunk size}}{B \times T}$. Formally, when a mean reduction is employed during the CrossEntropy loss calculation, the gradients are calculated for a particular input chunk and are not normalized over the entire input sequence. This additional scaling factor addresses such approximation issues.

3.3 Testing Best Practices

Testing is the cornerstone of our kernel development process. Exactness is non-negotiable, as even minor deviations can have far-reaching consequences. Through rigorous research and practical experience, we have distilled our approach into a set of best practices that ensure our kernels meet the highest standards of precision and reliability.

3.3.1 Correctness

Ensuring kernel precision is crucial, as any deviation from the original implementation could impact model convergence or cause critical errors. To achieve this, we prepare a pure PyTorch implementation (e.g., one provided by HuggingFace) for comparison and test the implementation with various input shapes and data types. We include regular shapes (e.g., powers of 2) and test irregular shapes to ensure proper handling of edge cases. We set appropriate absolute and relative tolerance levels: for fp32, use atol = 10^{-7} and rtol = 10^{-5}; for bf16, use atol = 10^{-3} and rtol = 10^{-2} [12].

Furthermore, large tensor dimensions can lead to inadvertent memory access issues. By default, the `program_id` in the kernels are stored as `int32`. If `program_id * Y_stride > 2,147,483,647`, the value becomes negative, resulting in illegal memory access. Such overflows and incorrect memory addressing errors can be avoided by explicitly converting it to `int64` when dealing with large dimensions.

3.3.2 Performance

We ensure that the re-implementation of kernels in Triton is justified (compared to the baseline version) by testing across two key dimensions: speed and memory usage.

For input shapes in testing, we use actual dimensions/hyper-parameters from the training process, such as a batch size of 4, a hidden dimension of 2048, and a variable sequence length. This approach ensures that the test results reflect expected gains in production training across a family of models.

3.3.3 Convergence Test

In practical training settings, the contiguity, shape, and dtype of tensors might differ from the unit test conditions. To prove the validity of our computational gains, we mimic such real-world scenarios at a smaller scale and verify the exactness of logits, weights, and loss at the end of the training.

3.3.4 Contiguity

Since Triton operates directly on physical memory, non-contiguous tensors (where elements are not arranged sequentially) can lead to illegal memory access or incorrect outputs. For example, when deploying our RoPE kernel for production training, we observed significant loss divergence because the derivative from the `scaled_dot_product_attention` function was not stored contiguously. To prevent such issues, it's best practice to ensure tensors are contiguous before passing them to the kernel.

3.4 Integrations

Liger has been successfully integrated with several popular training frameworks within the machine learning community, including Hugging Face transformers' `Trainer` class[13], Hugging Face TRL's `SFTTrainer` class[14], `Axolotl`[15], and `LLaMA-Factory`[16]. These integrations demonstrate the flexibility and ease of use of the Liger API, enabling developers to leverage its optimization capabilities with minimal code changes. A simple flag is typically all that is needed to patch the model code with Liger kernels. For example:

```
from trl import SFTConfig, SFTTrainer

trainer = SFTTrainer(
    "meta-llama/Meta-Llama-3-8B",
    train_dataset=dataset,
    # Setting 'use_liger=True' will load the model using AutoLigerKernelForCausalLM
    args=SFTConfig(..., use_liger=True),
)
trainer.train()
```

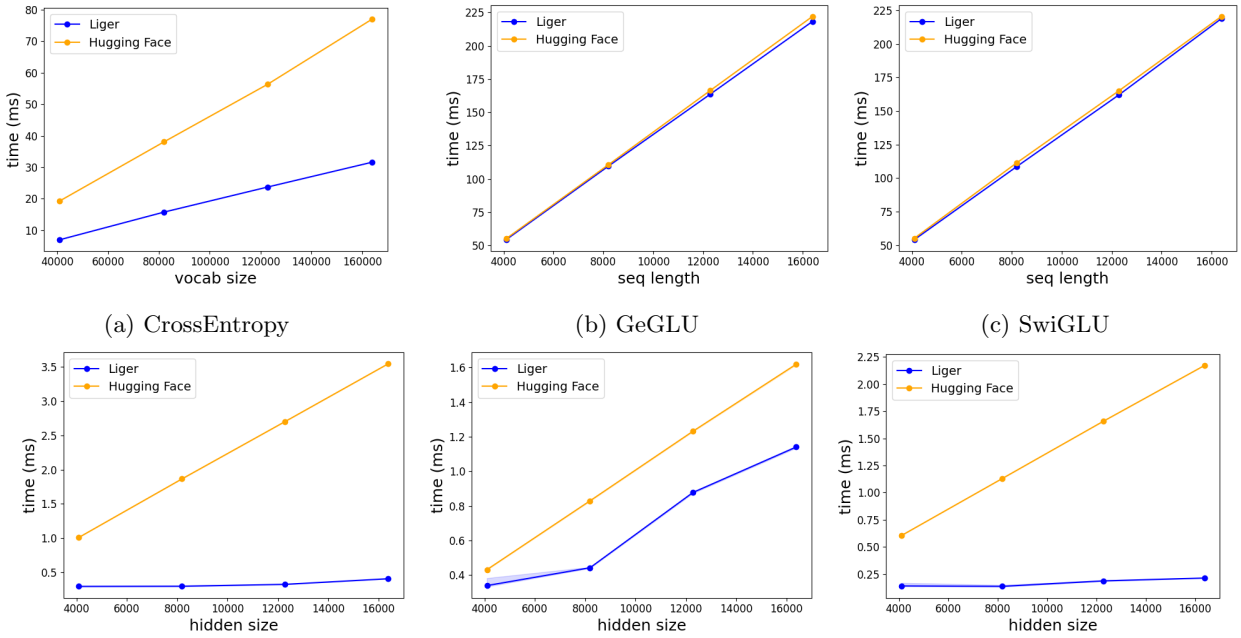
[12] Note that in practice, the tolerance may need further relaxation in some cases by one or two orders of magnitude, even for exact kernels. We use convergence tests to ensure exactness in cases where the tolerance for correctness needs to be loose.

[13] https://huggingface.co/docs/transformers/en/main_classes/trainer

[14] https://huggingface.co/docs/trl/main/en/sft_trainer

[15] https://axolotl-ai-cloud.github.io/axolotl/#liger-kernel

[16] https://github.com/hiyouga/LLaMA-Factory



(a) CrossEntropy (b) GeGLU (c) SwiGLU

(d) RMSNorm (e) LayerNorm (f) RoPE

Figure 2: Kernel execution speed benchmarks.

4 Numerical Experiments

This section presents the kernel level and end-end LLM training benchmarks using `Liger-Kernel` v0.2.1[17].

4.1 Kernel Benchmark

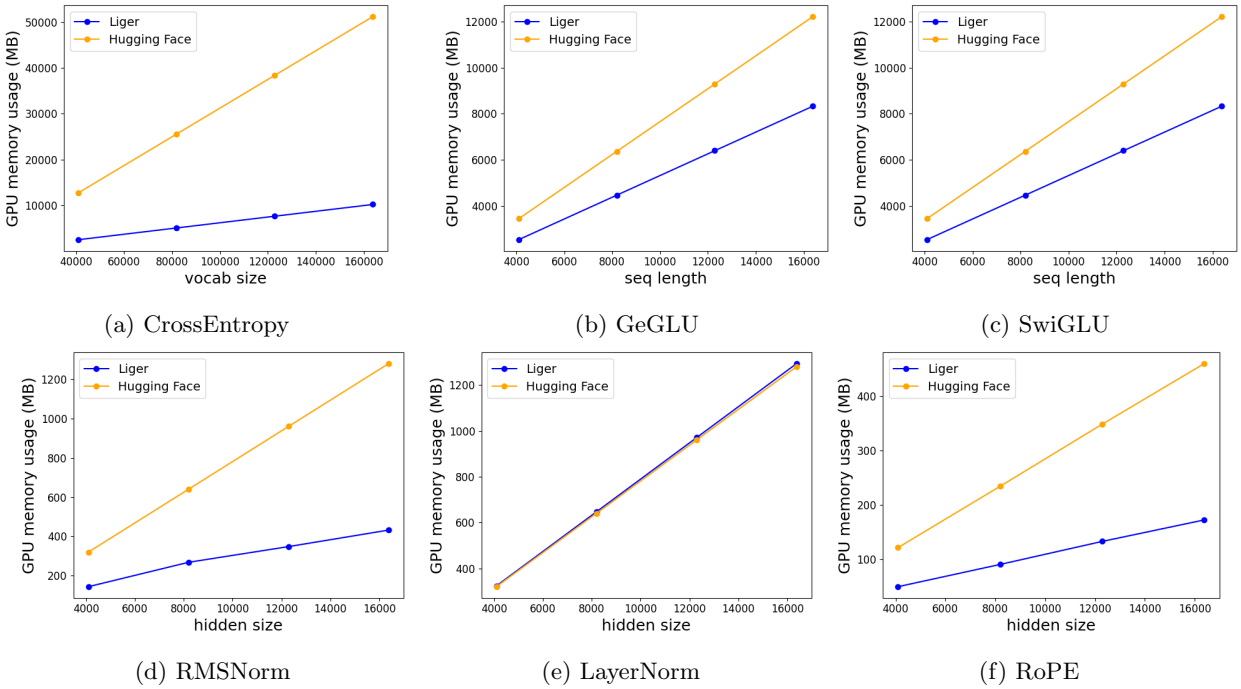
We benchmark the kernels individually across a variety of settings and illustrate the improvements in speed and memory consumption with Liger.

Setup. All benchmarks are run on a single NVIDIA A100 GPU (80 GB). The CrossEntropy kernel is benchmarked on vocab sizes in the set $\{40960, 81920, 122880, 163840\}$. The GeGLU and SwiGLU kernels are benchmarked on varying sequence lengths, whereas the RMSNorm, LayerNorm, and RoPE kernels are benchmarked on varying hidden dimensions. The sequence lengths and hidden dimension sizes are chosen from $\{4096, 8192, 12288, 16384\}$. All benchmarks are repeated 10 times to plot the median speed and memory along with $[0.2, 0.8]$ quantile values as the lower and upper bounds.

Results. The kernel speed and memory benchmarks are illustrated in Figure 2, 3 respectively. Observe that all the Liger-kernel implementations either execute faster, consume less memory or provide both of these benefits when compared to the baseline implementations. In the case of the CrossEntropy kernel, the online softmax computation along with in-place replacement of the kernel inputs with their gradients leads to approximately 3× faster execution (Figure 2a) and consumes approximately 5× less memory (Figure 3a) for a vocab size of 163840. For GeGLU and SwiGLU, we maintain parity with the baseline in terms of speed (Figure 2b, 2c) and reduce the peak memory consumption by roughly 1.6× (when sequence length is 16384) by recomputing the SiLU$(\cdot)$ and GELU$(\cdot)$ outputs during the backward pass (Figure 3b, 3c).

The RMSNorm implementation fuses the normalization and scaling operations into a single triton kernel and caches the root mean square values for usage in the backward pass. This avoids repetitive data transfers and floating point operations with minimal memory overheads. Figure 2d, 3d illustrates approximately 7×

[17] https://github.com/linkedin/Liger-Kernel/releases/tag/v0.2.1



(a) CrossEntropy (b) GeGLU (c) SwiGLU

(d) RMSNorm (e) LayerNorm (f) RoPE

Figure 3: Kernel peak allocated memory benchmarks.

reduction in execution time and roughly 3× reduction in peak memory consumption for a hidden dimension of 16384 respectively. A similar caching approach for the inverse root mean square is employed for LayerNorm kernel which results in approximately 30% reduction in execution time (Figure 2e) with minimal memory overheads (Figure 3e). Finally, for the RoPE kernel, we employ a flattened 1D tensor to represent the rotation matrix and leverage the repeated blocks in $\boldsymbol{R}^d_{\Theta,m}$ to significantly reduce the growth in latency with an increase in hidden dimension size. In particular, we achieve approximately 8× speedup with approximately 3× lower memory consumption for a hidden size of 16384.

4.2 Usecase Benchmark

Setup. For the end-end training experiments, we employ 4 NVIDIA A100 GPUs (80 GB each) to fine-tune the LLMs (LLaMA 3-8B, Qwen2, Gemma, Mistral, and Phi3) on the Alpaca dataset. We vary the batch size, set the precision to bfloat16, and use the AdamW optimizer with a cosine learning rate scheduler. The sequence length for training is set to 512 tokens. The throughput and GPU memory usage metrics are collected after 20 training steps with the standard error measured from 5 repetitive runs. The benchmark script can be found in our GitHub repository[18].

Performance Comparison. At a batch size of 64, LLaMA 3-8B demonstrates a **42.8% increase in throughput**, coupled with a **54.8% reduction in GPU memory usage** (Figure 4). This enables training on smaller GPUs or using larger batch sizes and longer sequence lengths with lower resource consumption. Similarly, at a batch size of 48 our kernels improve the throughput of Qwen2 by **25.5%**, while achieving a **56.8% reduction in GPU memory usage** (Figure 5). For Gemma, throughput improves by **11.9%** with a **51.8% reduction in memory usage** at a batch size of 48 (Figure 6). Mistral, at a batch size of 128, exhibits a **27% increase in throughput**, with a **21% drop in GPU memory usage** (Figure 7). Finally, Phi3, at a batch size of 128, shows a **17% increase in throughput**, while reducing memory usage by **13%** (Figure 8). Overall, the results highlight several notable use cases. LLaMA 3-8B's exceptional improvements make it ideal for resource-constrained environments where GPU memory is a bottleneck. Additionally,

[18] https://github.com/linkedin/Liger-Kernel/tree/main/examples/huggingface

Qwen2's strong memory reductions position it well for tasks involving large datasets or extended training durations. Mistral's high throughput gains make it advantageous for workloads requiring large batch sizes.

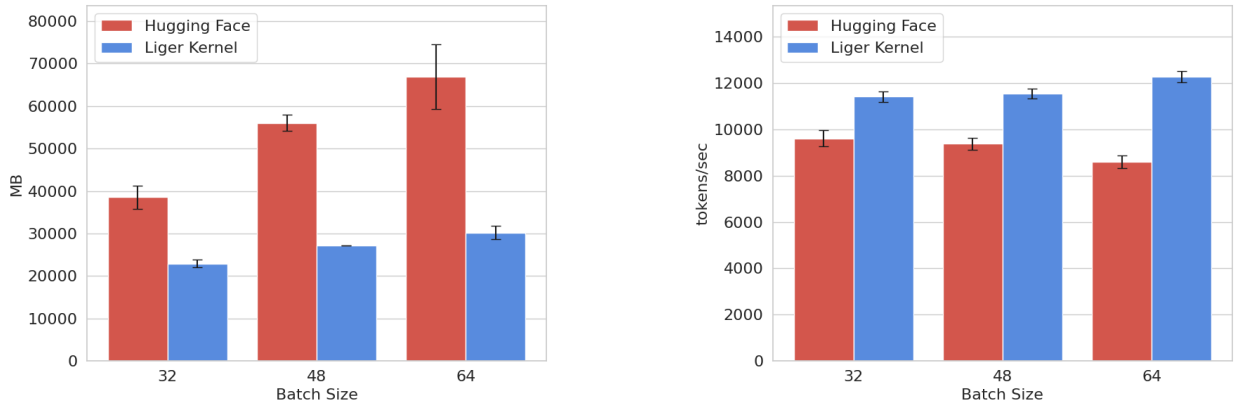


Figure 4: Comparison of peak allocated memory and throughput for LLaMA 3-8B.

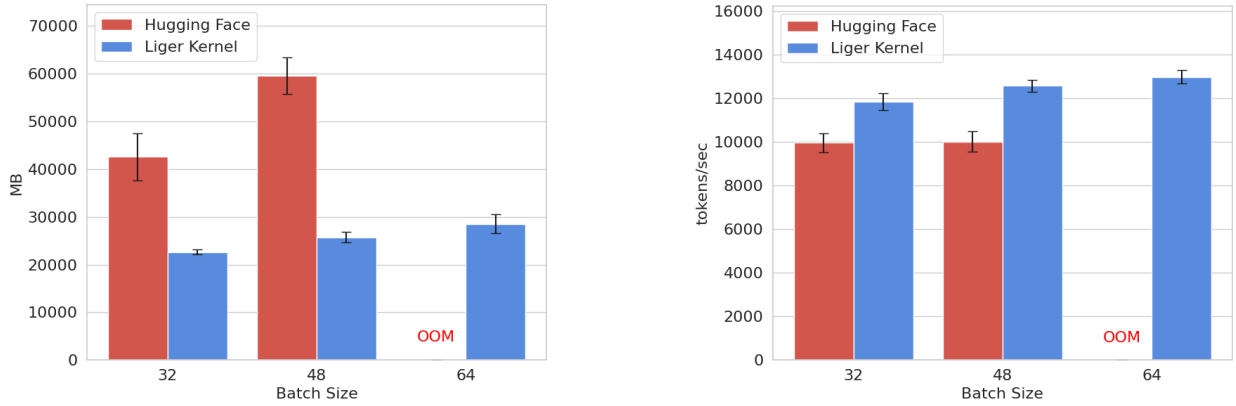


Figure 5: Comparison of peak allocated memory and throughput for Qwen2.

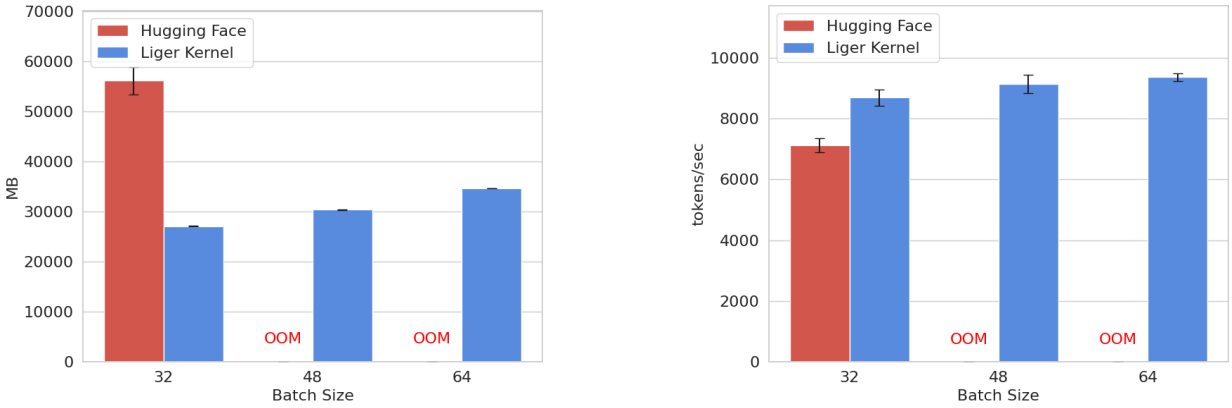


Figure 6: Comparison of peak allocated memory and throughput for Gemma 7b.

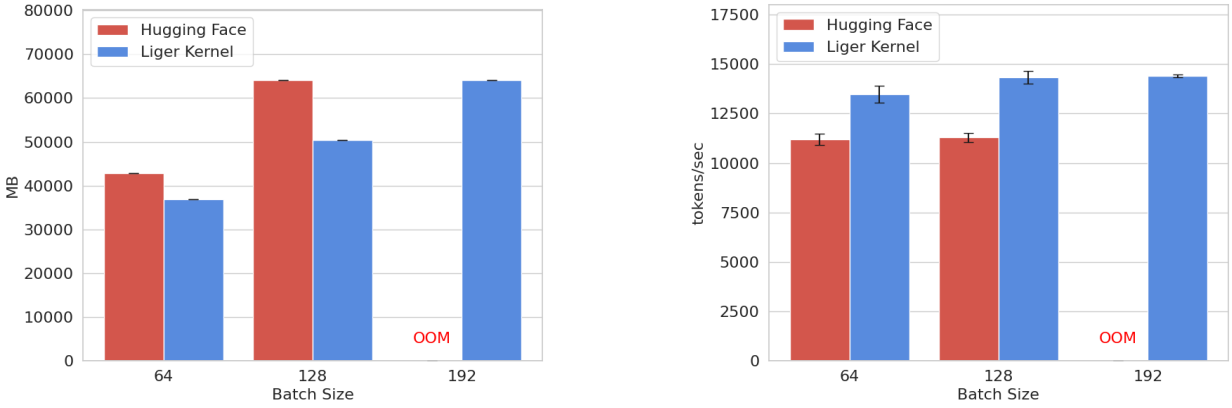


Figure 7: Comparison of peak allocated memory and throughput for Mistral 7b.

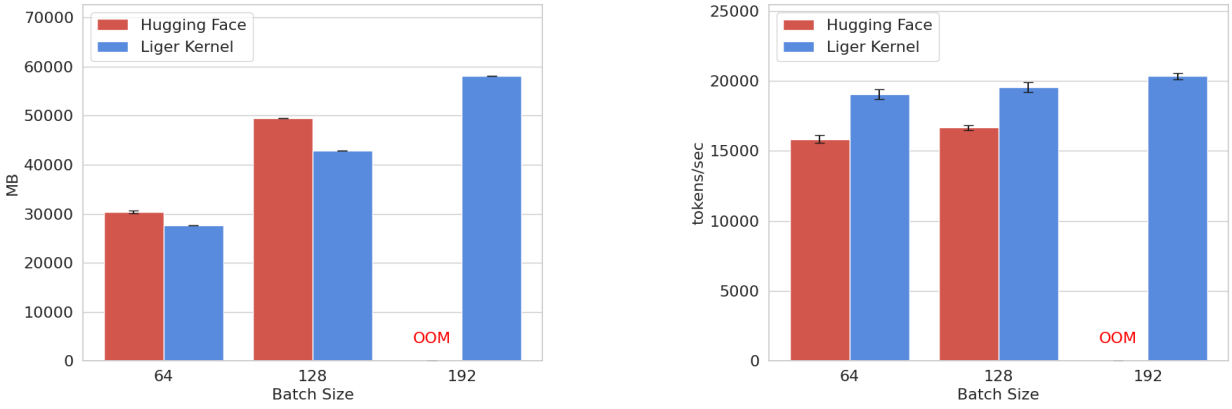


Figure 8: Comparison of peak allocated memory and throughput for Phi3.

Medusa. Medusa (Cai et al., 2024) is a simple framework that democratizes acceleration techniques for LLM generation by using multiple decoding heads to predict several subsequent tokens in parallel. During training, Medusa requires adding k decoding heads to the hidden states right before the regular LM head h_t. The k-th head is used to predict the token in the $(t+k+1)$-th position of the next tokens (the original language model head is used to predict the $(t+1)$-th position).

The Liger LFCE kernel is particularly effective in this context, as it eliminates the need to materialize logits for each decoding head. This is critical in scenarios with large vocabulary sizes, such as LLaMA-3's 128k tokens, where materializing logits can lead to significant memory consumption. The introduction of multiple decoding heads often results in out of memory issues. However, by leveraging the Liger fused CE kernel, which computes gradients in place without materializing logits, we achieve highly efficient results. This approach enables further exploration and development in multi-token prediction.

Medusa training has two flavors. The first, called stage-1, involves training only the additional Medusa heads while keeping the backbone LLM frozen. The second approach tunes both the backbone and the LLM heads simultaneously. We have benchmarked both cases, and the Liger kernel has demonstrated reduced memory usage and improved throughput. Without the Liger kernel, experiments are highly prone to out of memory issues. In Figures 9-12, the standard errors measured from repetitive runs are typically less than 1% hence not visible from most of the plots.

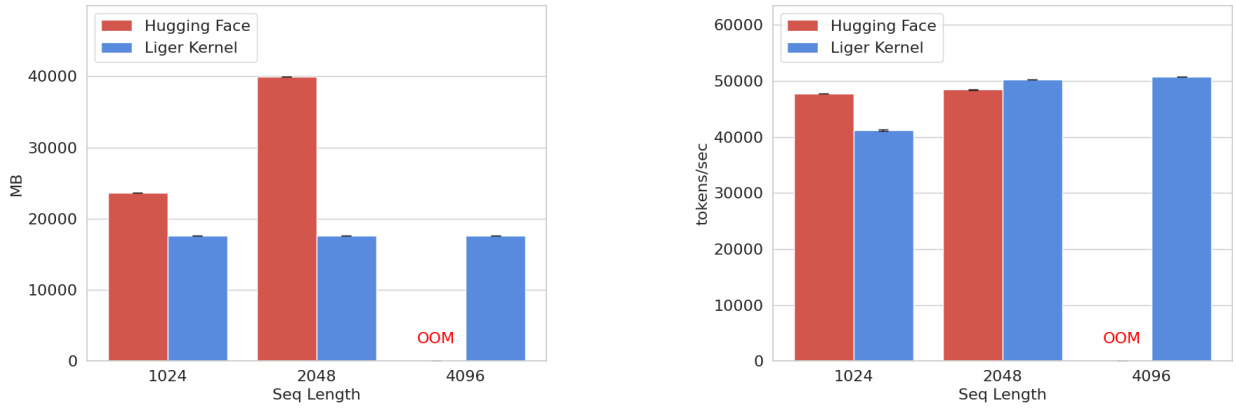


Figure 9: Comparison of peak allocated memory and throughput for Stage 1 with 3 Medusa heads.

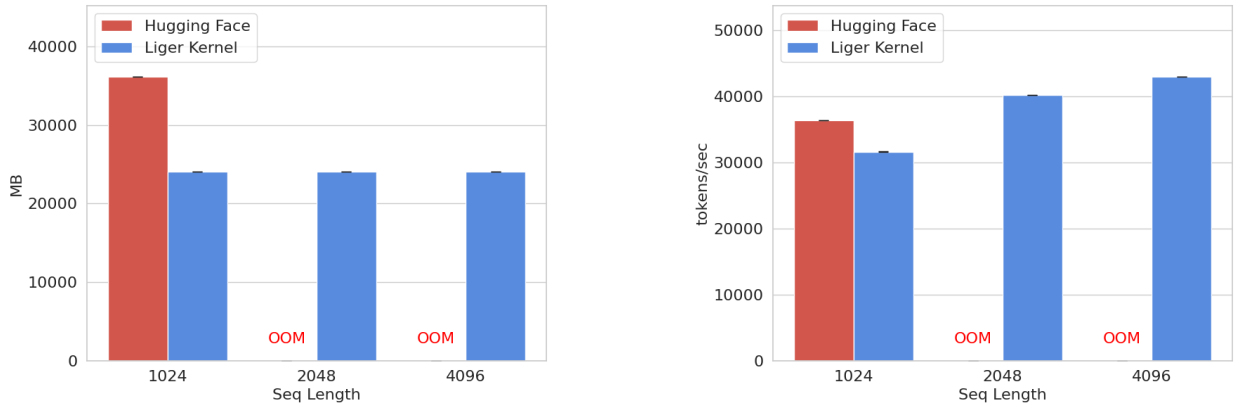


Figure 10: Comparison of peak allocated memory and throughput for Stage 1 with 5 Medusa heads.

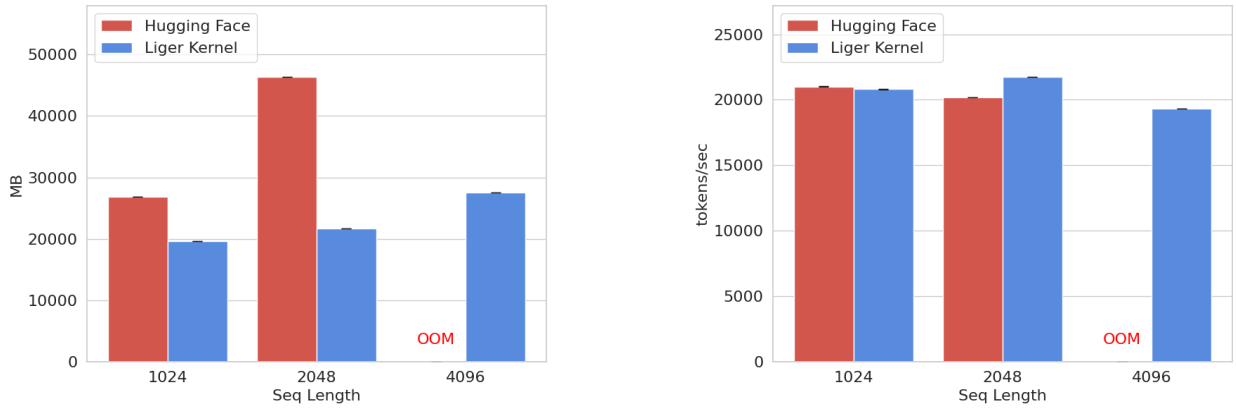


Figure 11: Comparison of peak allocated memory and throughput for Stage 2 with 3 Medusa heads.

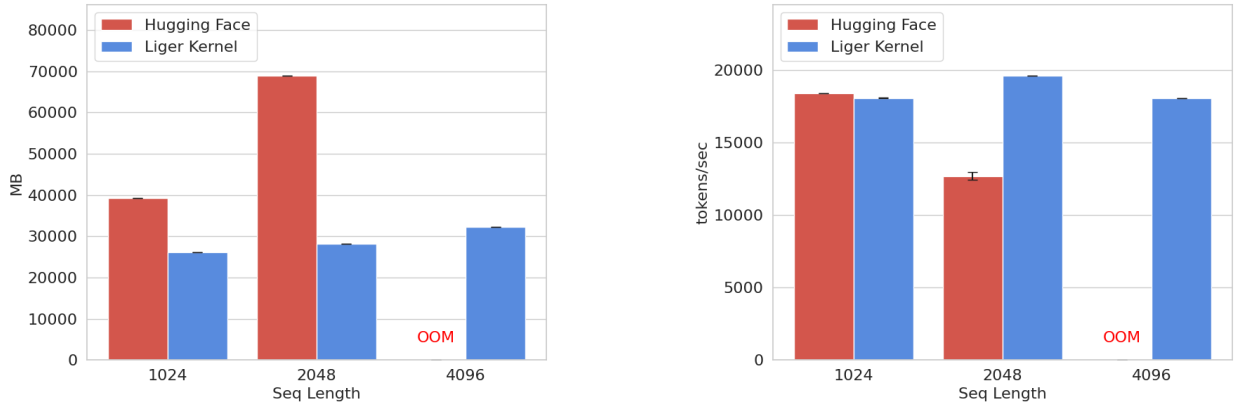


Figure 12: Comparison of peak allocated memory and throughput for Stage 2 with 5 Medusa heads.

Note: This technical report focuses solely on performance benchmarking. Generating effective LLM heads that can accelerate inference for the LLaMA3-8B model is not within the scope of this report. Such work requires extra work for training data selection, hyperparameter tuning, and warmup techniques to ensure proper model convergence. Our experiments utilize 8 NVIDIA A100 GPUs (80 GB each) to train the LLaMA 3-8B model with a variable sequence length, a batch size of 4, bfloat16 precision and the AdamW optimizer.

5 Conclusions

Liger Kernel offers optimized Triton kernels that improve training efficiency with a user-friendly API, seamless integration with popular frameworks, and a commitment to performance. Our goal is to make Liger Kernel the leading open-source Triton kernel library for LLM training. We aim to achieve this by focusing on:

- **Ease of Use:** Offering intuitive APIs, broad model support, and wide hardware compatibility
- **Performance Focus:** Maximizing computational efficiency and ensuring exactness.
- **Ecosystem Engagement:** Building a strong community through events and collaborations with industry leaders, alongside fostering recognition and branding for contributors.
- **Operational Excellence:** Ensuring stable CI, rigorous testing protocols, and an active community.

With these commitments, `Liger-Kernel` aspires to become the preferred choice for efficient and scalable LLM training, driving innovation and adoption within the deep learning community. While existing work primarily focuses on training, the same techniques can be seamlessly adapted for optimizing model inference.

6 Contributors and Acknowledgements

6.1 Core Contributors

Pin-Lun Hsu Project lead. Led, architected, and implemented multiple kernels, public interface, and test suite.
Yun Dai Core contributor. Designed an efficient version of RoPE, GeGLU, and improved the precision of Fused Linear CrossEntropy. Designed the public interface.
Vignesh Kothapalli Core contributor. Implemented Fused Linear CrossEntropy and designed the scaling and sharding formula.
Qingquan Song Core contributor. Implemented SwiGLU. Led the convergence tests and PyTorch lightning integration. Ensure the contiguity of RoPE and kernel testing precisions.
Shao Tang Core contributor. Implemented Layer Norm variants. Derived gradient formulas for different cases. Proposed best kernel practices, including ensuring contiguity and conducting convergence tests.
Siyu Zhu Core contributor. Implemented Fused Linear CrossEntropy and adapted the kernel for the Medusa (multi-token prediction) use case, proving its effectiveness with benchmarks. Led the Hugging Face integration.
Steven Shimizu Contributor. Improved HuggingFace integration and contributed to the tests.
Shivam Sahni Contributor. Expanded model support and made several kernel improvements.
Haowen Ning Contributor and the overall team lead of LLM training infra.
Yanning Chen Contributor and the team manager.

6.2 Acknowledgement

We thank AMD and Intel for funding GPUs for our AMD and Intel CI. We also thank Modal for funding 3000 credits from GPU MODE IRL for our NVIDIA CI.

We thank Triton[19], flash-attention[20], and Unsloth[21] for the reference of Triton kernels for LLM training, tiny shakespeare dataset[22] and llm.c[23] for convergence testing design, Efficient Cross Entropy[24] for fused linear cross entropy reference, AutoAWQ[25] and **Robert Shaw** for Automodel design, as well as Hugging Face, PyTorch Lightning, Axolotl, and Llama-Factory for the collaboration.

We also thank our leaders **Animesh Singh** and **Kapil Surlaker** for their invaluable expertise in the ML infrastructure stack and open-source strategy.

Thanks to **Claire (Yi-Shan) Wu** for the LOGO design and Wave Snippets[26] for generating the animated code snippets.

References

Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. {TensorFlow}: a system for {Large-Scale} machine learning. In *12th USENIX symposium on operating systems design and implementation (OSDI 16)*, pages 265–283, 2016.

[19] https://triton-lang.org/main/getting-started/tutorials/index.html
[20] https://github.com/dao-ailab/flash-attention
[21] https://github.com/unslothai/unsloth
[22] https://huggingface.co/datasets/karpathy/tiny_shakespeare
[23] https://github.com/karpathy/llm.c
[24] https://github.com/mgmalek/efficient_cross_entropy
[25] https://github.com/casper-hansen/AutoAWQ
[26] https://www.wavesnippets.com/

Marah Abdin, Sam Ade Jacobs, Ammar Ahmad Awan, Jyoti Aneja, Ahmed Awadallah, Hany Awadalla, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Harkirat Behl, et al. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219*, 2024.

Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, et al. Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 929–947, 2024.

Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *stat*, 1050:21, 2016.

Tom B Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Proceedings of the 34th International Conference on Neural Information Processing Systems*, pages 1877–1901, 2020.

Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*, 2024.

Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, et al. TVM: An automated End-to-End optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 578–594, 2018.

Yun Dai, Tejas Dharamsi, Byron Hsu, Tao Song, and Hamed Firooz. Enhancing stability for large models training in constrained bandwidth networks. *arXiv preprint arXiv:2407.01614*, 2024.

Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, 2023.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *arXiv preprint arXiv:2205.14135*, 2022.

Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

Roy Frostig, Matthew James Johnson, and Chris Leary. Compiling machine learning programs via high-level tracing. *Systems for Machine Learning*, 4(9), 2018.

Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*, 2016.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.

Benjamin Lefaudeux, Francisco Massa, Diana Liskovich, Wenhan Xiong, Vittorio Caggiano, Sean Naren, Min Xu, Jieru Hu, Marta Tintore, Susan Zhang, Patrick Labatut, Daniel Haziza, Luca Wehrstedt, Jeremy Reizenstein, and Grigory Sizov. xFormers: A modular and hackable transformer modelling library. https://github.com/facebookresearch/xformers, 2022.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.

Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 3505–3506, 2020.

Amit Sabne. XLA : Compiling machine learning for peak performance, 2020.

Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

J Su, Y Lu, S Pan, A Murtadha, B Wen, and Y Liu Roformer. Enhanced transformer with rotary position embedding., 2021. *DOI: https://doi. org/10.1016/j. neucom*, 2023.

Gemini Team, Rohan Anil, Sebastian Borgeaud, Yonghui Wu, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.

Philippe Tillet, Hsiang-Tsung Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pages 10–19, 2019.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

A Vaswani. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.

Guanhua Wang, Heyang Qin, Sam Ade Jacobs, Connor Holmes, Samyam Rajbhandari, Olatunji Ruwase, Feng Yan, Lei Yang, and Yuxiong He. Zero++: Extremely efficient collective communication for giant model training. *arXiv preprint arXiv:2306.10209*, 2023.

Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, et al. Emergent abilities of large language models. *Transactions on Machine Learning Research*, 2022.

W Hwu Wen-Mei, David B Kirk, and Izzat El Hajj. *Programming Massively Parallel Processors: A Hands-on Approach*. Morgan Kaufmann, 2022.

Biao Zhang and Rico Sennrich. Root mean square layer normalization. *Advances in Neural Information Processing Systems*, 32, 2019.

Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, et al. Pytorch FSDP: Experiences on scaling fully sharded data parallel. *Proceedings of the VLDB Endowment*, 16(12):3848–3860, 2023.